

Airflow at the center of your pipelines

Is your data development environment slowing you down? Reclaim the "Edit-Run-Debug" agility without the infrastructure overhead. ⚡

The context

If you work with Apache Airflow, you know the invisible bottleneck: environment friction. Setting up containers, waiting for the Scheduler to pick up your DAGs, and dealing with the weight of a "Cloud-Native" stack on your local machine can drain any engineering team's productivity. Modern orchestration shouldn't require a server cluster just to validate transformation logic.

Why engineers shift to wpipe

- Pipelines are Python: versionable, testable, reviewable in Git.
- State persisted to SQLite WAL at every step; resume, don't restart.
- <50MB RAM. No broker, no daemon, no metadata server.

Code-first

Pipelines are Python: versionable, testable, reviewable in Git.

Checkpoints

State persisted to SQLite WAL at every step; resume, don't restart.

Independently lean

<50MB RAM. No broker, no daemon, no metadata server.

What it looks like

```
from wpipe import Pipeline, step

@step(name="task", retry_count=3, retry_delay=2)
def task(data):
    return {"ok": run(data["job"])}

pipe = Pipeline(pipeline_name="airflow_like", tracking_db="jobs.db")
pipe.set_steps([task])

# No scheduler daemon, no heavy metadata DB.
```

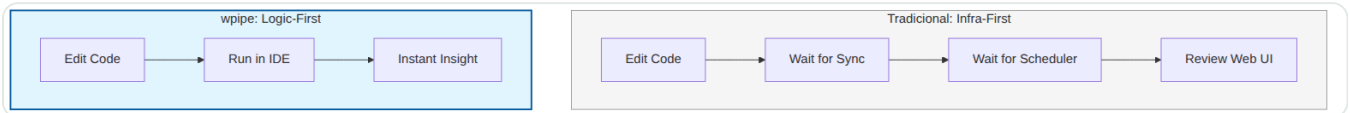
At a glance

Development Metric	Apache Airflow	wpipe (Modern Lib)
Local Setup	Complex (Docker/Helm)	Instant (`pip install`)
Feedback Loop	Slow (Scheduler Latency)	Immediate (Direct Run)

Development Metric	Apache Airflow	wpipe (Modern Lib)
Resource Usage	GBs of RAM	MBs of RAM
Auditability	Centralized logs	Native Local SQLite Tracker
Maintainability	High (Infrastructure)	Low (Code Only)

Under the hood

The flow behind the post, as wpipe renders it from code.



+117k downloads
A community of efficiency-first developers.

<50MB RAM
Fits containers, VMs, and Raspberry Pi-class hardware.

SQLite WAL
Industrial durability without a database to administer.

Next step

wpipe is a Python library — try it in your next pipeline:

```
pip install wpipe
```

Steps stay functions you already understand. Docs, diagrams, and dashboards follow. Explore **Airflow** workflows at wpipe.readthedocs.io.

wpipe · Python-native orchestration with SQLite WAL checkpoints, retries, timeouts, parallelism, and self-documenting flows. +117k downloads.